

6교시: Compose 장애 분석

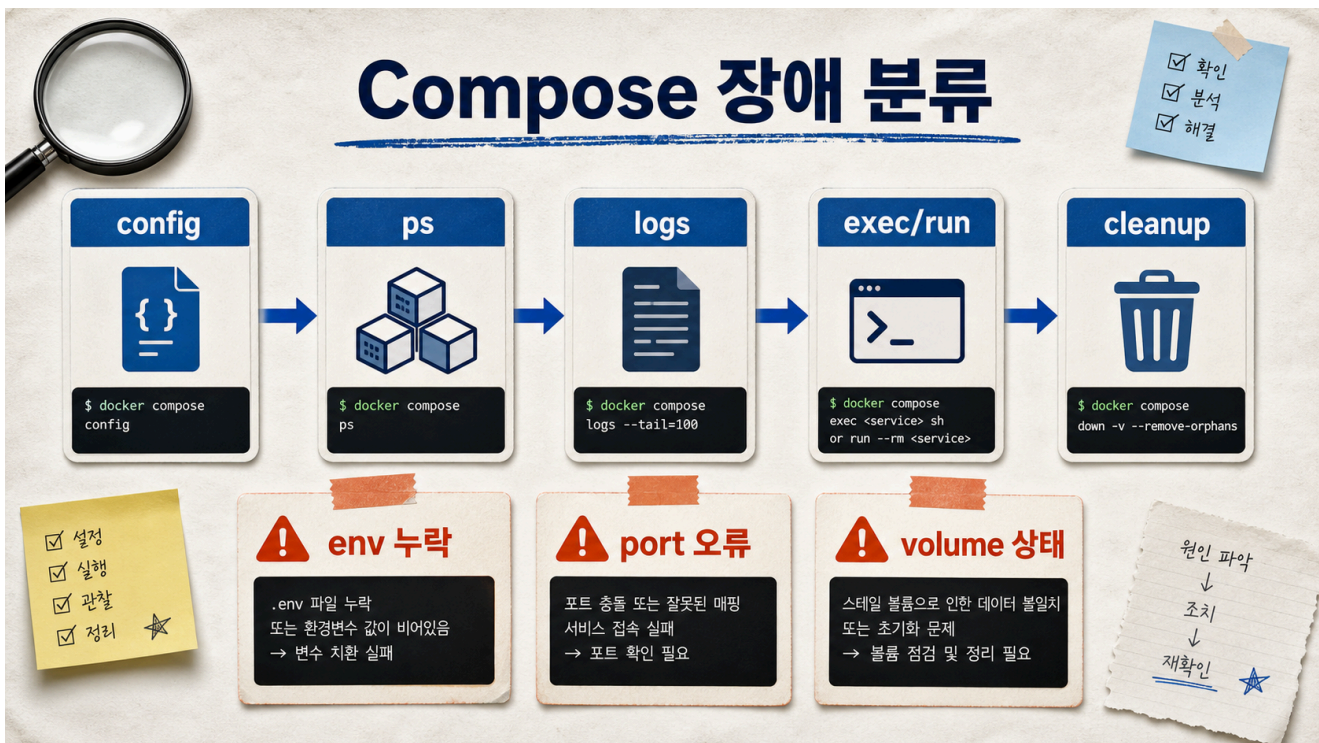
수업 목표

- Compose 장애를 config, start, runtime, cleanup 단계로 분류한다.
- missing env, wrong port, stale volume을 재현하고 evidence를 남긴다.
- config -> ps -> logs -> run/exec -> inspect 순서로 확인한다.

50분 흐름

시간	활동	비중	산출
0-8분	장애 분류 기준 소개	설명 15%	triage map
8-20분	missing env drill	실행 25%	config error
20-30분	wrong port drill	실행 20%	HTTP failure note
30-40분	volume 초기화 이슈 설명	설명 20%	data lifecycle note
40-50분	RCA 기록 작성	실행 20%	failure record

Visual 1: Compose troubleshooting order



이 visual은 장애를 볼 때의 순서를 보여준다. 볼 지점은 처음부터 container 내부로 들어가지 않고 config와 service 상태를 먼저 확인한다는 점이다.

missing env drill

.env에서 POSTGRES_PASSWORD 줄을 제거한 뒤 실행한다.

```
cd week2/day4/labs/compose-app
docker compose config
```

기대 실패:

```
set POSTGRES_PASSWORD in .env
```

복구:

```
cp .env.example .env
```

wrong port drill

```
docker compose up -d
curl -I http://localhost:80
curl -I http://localhost:18084
docker compose ps
```

host 80이 실패하고 18084가 성공하면 web container 문제가 아니라 host 접근 port 문제로 분류한다.

stale volume 설명

PostgreSQL official image는 data directory가 이미 초기화되어 있으면 init script를 다시 실행하지 않는다. db/init/001_schema.sql을 바꿨는데 DB 안의 결과가 바뀌지 않으면 container 문제가 아니라 기존 named volume state 문제일 수 있다.

초기화가 필요한 실습 cleanup:

```
docker compose down -v
docker compose up -d
```

운영 주의:

- down -v는 DB data 삭제다.
- runbook에 넣을 때는 "실습 초기화 전용"이라고 명시한다.

RCA 템플릿

```
## Compose Failure RCA
```

- Symptom:
- Reproduce:
- Observe:
- Hypothesis:
- Verification:
- Fix:
- Recheck:
- Prevention:

평가 기준

기준	2점 evidence
분류	config/start/runtime/cleanup 중 어디인지 구분했다
재현	failure drill 명령과 증상을 기록했다
복구	수정 후 재확인 evidence를 남겼다

공식 근거 링크

- Compose services reference: <https://docs.docker.com/reference/compose-file/services/>
- PostgreSQL Docker Official Image: https://hub.docker.com/_/postgres

선행 지식과 범위 경계

이 교시는 장애를 빨리 고치는 시간이라기보다 장애를 올바르게 분류하는 시간이다. 초급자는 오류를 보면 바로 명령을 바꾸거나 container를 지우려 하지만, DevOps 실무에서는 먼저 증거를 모으고 failure domain을 좁힌다.

Day 4의 장애 범위는 local Compose project 안으로 제한한다. cloud network, Kubernetes service, managed database 장애는 다루지 않는다. 그러나 분류 방식은 이후 주차에서도 그대로 사용된다.

학술 기준 연결

Google SRE의 postmortem culture는 blame보다 learning을 강조한다. 수업의 RCA도 "누가 잘못했는가"가 아니라 "어떤 증거로 원인을 좁혔는가"를 본다.

Bloom taxonomy 기준으로 이 교시는 analysis와 evaluation에 해당한다. 학생은 로그와 상태를 보고 config 문제인지, startup 문제인지, runtime dependency 문제인지 평가해야 한다.

장애 분류 모델

Compose 장애는 다음 네 단계로 나누면 빠르게 좁힐 수 있다.

단계	대표 명령	실패 예
config	<code>docker compose config</code>	YAML 오류, env 누락
create/start	<code>docker compose up -d, ps</code>	image pull 실패, port bind 실패
runtime	<code>logs, exec, run</code>	DB readiness, service DNS, app error
cleanup/state	<code>down, volume ls</code>	stale volume, data 삭제

이 순서를 지키면 불필요하게 container를 반복 삭제하는 시간을 줄일 수 있다.

missing env RCA 예시

```
## Compose Failure RCA
- Symptom: `docker compose config` fails before containers start.
- Impact: web/db project cannot be started.
- Reproduce: remove `POSTGRES_PASSWORD` from `.env`.
- Observe: error message includes `set POSTGRES_PASSWORD in .env`.
- Hypothesis: required variable interpolation failed.
- Verification: restore variable and run `docker compose config`.
- Fix: update `.env.example` and README setup step.
- Recheck: config succeeds and db service environment is rendered.
- Prevention: keep required variable syntax in Compose file.
```

이 예시는 container log가 아니라 config 단계에서 끝난 장애다. 그러므로 `docker compose logs`를 먼저 볼 필요가 없다.

wrong port RCA 예시

```
## Wrong Port RCA
- Symptom: browser cannot open `http://localhost:80`.
- Observe: `docker compose ps` shows `0.0.0.0:18084->80/tcp`.
- Root cause: user accessed host port 80 instead of published host port 18084.
- Fix: use `http://localhost:18084` or change `WEB_PORT`.
- Recheck: `curl -I http://localhost:18084` returns HTTP 200.
- Prevention: README must state host port and container port separately.
```

여기서 nginx 내부 port를 바꾸면 문제 해결이 아니라 혼란을 키울 수 있다. 실패 위치는 host 접근 port 다.

stale volume RCA 예시

PostgreSQL image는 data directory가 비어 있을 때만 init script를 실행한다. 따라서 SQL 파일을 수정해도 기존 volume이 남아 있으면 반영되지 않을 수 있다.

```
## Stale Volume RCA
- Symptom: changed init SQL is not reflected in database.
- Observe: db starts successfully, but expected row/table is missing.
- Hypothesis: existing named volume already contains initialized database.
- Verification: `docker volume ls --filter name=compose-app`.
- Fix for practice reset: `docker compose down -v`, then `up -d`.
- Risk: `down -v` deletes DB data.
- Prevention: document migration vs reset separately.
```

실무 사고방식

장애 분석에서 가장 위험한 습관은 "일단 다 지우고 다시 실행"이다. 실습에서는 빠를 수 있지만, 운영에서는 증거를 잃고 데이터를 삭제할 수 있다.

실무에서는 다음 순서를 선호한다.

1. 증상 기록 2. 영향 범위 기록 3. 변경 사항 확인 4. 로그와 상태 수집 5. 원인 후보 작성 6. 가장 작은 수정 적용 7. 재확인 8. 예방책 문서화

위험 분류표

장애	Severity	이유	완화
missing env	Medium	실행 전 실패, data 손상 없음	config check
wrong port	Low	접근 실패지만 service는 정상	ps와 README
stale volume	High	data 삭제 위험과 연결	reset 명령 분리
DB health fail	Medium	dependent service 영향	logs와 healthcheck
secret 노출	High	repository credential risk	placeholder와 ignore

오해 점검 문항

문항	기대 답
모든 장애는 logs부터 보면 되는가	아니다. config 단계 장애는 logs가 없을 수 있다
down -v는 안전한 cleanup인가	data 삭제가 포함될 수 있어 위험하다
port 접속 실패는 항상 web container 문제인가	아니다. host port 오해일 수 있다
init SQL 수정 후 DB가 안 바뀌면 image 문제인가	기존 volume 상태 문제일 수 있다

Evidence 수준 구분

수준	예시
약한 evidence	"에러 해결함"
중간 evidence	에러 메시지와 수정 명령 기록
강한 evidence	symptom, impact, hypothesis, fix, recheck, prevention까지 기록

전이 과제

학생은 오늘 겪은 장애 또는 일부러 재현한 장애 하나를 골라 RCA를 작성한다. 반드시 포함할 항목은 다음과 같다.

- 실패 명령
- 실제 출력 또는 핵심 로그
- 원인 후보 2개 이상
- 확인한 증거
- 수정한 내용
- 재확인 명령
- 다음 사람을 위한 예방 문장